

DATA 550 Lab 1 - Visualizing world health data

There are two versions of this lab, one in Python and one in R. The R lab will use `ggplot` and the Python lab will use `Altair`.

This is the Python version.

Each partner should choose a version to complete, though keep in mind that **you are required to alternate between completing the R labs and the Python labs** to get experience using both languages.

Acknowledgements

This lab was lightly adapted from the one created by Joel Ostblom for the DSCI 531 version of Data Viz I in the 2020/21 Academic year.

Submission instructions

rubric={mechanics:2}

You receive marks for submitting your lab correctly, please follow these instructions:

- Each partner should complete one file, though collaboration is permitted.
- Push your `.ipynb` files to GitHub frequently (i.e. not just in one fell swoop at the end).
- Don't forget to submit a clickable link to your GitHub repo on Canvas so we can grade it.

```
In [1]: # Run this cell to ensure that altair plots show up in the exported HTML
import altair as alt

# Save a vega-lite spec and a PNG blob for each plot in the notebook
# This is needed **only** to make sure plots render as images on GitHub
# Unfortunately this disables interactivity, so perhaps comment this
alt.renderers.enable('mimetype')

# Handle large data sets without embedding them in the notebook
alt.data_transformers.enable('vegafusion')
```

```
import warnings
warnings.filterwarnings('ignore')

alt.renderers.enable('html')
```

```
Out[1]: RendererRegistry.enable('html')
```

1. Get motivated!

You may have seen the Gapminder world health data set in the past and we will revisit an updated version of it in this lab. The Gapminder foundation strives to educate people about the public health status in countries all around the world and fight devastating misconceptions that hinder world development. This information is important both for our capacity to make considerate choices as individuals, and from an industry perspective in understanding where markets are emerging. In their research, Gapminder has discovered that most people don't really know what the world looks like today. Do you? [Take this 7-8 min quiz to find out.](#)

This quiz is not easy, so don't worry if you get a low score. It is primarily meant to spark your curiosity to learn more about this lab's data set!

Question 1.1

rubric={reasoning:1,writing:1}

To answer the first lab question [watch this 20 min video of Hans Rosling](#) a public health professor at Karolinska Institute who founded Gapminder together with his son and his son's wife. Although the video is almost 15 years old, it is a formidable demonstration on how to present data in a way that engages your audience while conveying a strong, important message. (The original clip has over 3 million views, but I linked you one of better video quality).

Briefly describe (≤ 90 words) what you think is the most important message conveyed in the video and which data visualization you think was the most effective in getting this message across to the viewers.

Hans Rosling's main point is that the old "West vs. Rest" divide is dead. Most of the world now sits right in the middle.

He used those famous animated bubble charts to prove it, mapping life expectancy against family size over two centuries. Watching the bubbles move makes it obvious that we are all converging toward longer, healthier lives. It turns complex data into an intuitive story of global progress, replacing outdated stereotypes with a much more accurate and hopeful reality

2. The Gapminder bubble chart

The "bubble chart" have become quite famous from their appearance in the Gapminder talks, and are widely used in other areas as well. Let's start by recreating a simple version of this chart ourselves!

There will be some data wrangling involved in this lab, and since this course is primarily about visualization and this is the first lab, I will give you some hints for most data wrangling parts of this lab. Often I will link documentation or StackOverflow, so that you get practice finding information on these sources, and sometimes you will need to search them yourself if I haven't included a link.

To make this more interesting, I have compiled a more recent version of the Gapminder dataset, which contains values up until 2018 for most of the features. We will not use all the columns in the data set, but here is a description of what they contain that you can refer back to throughout the lab.

Column	Description
country	Country name
year	Year of observation
population	Population in the country at each year
region	Continent the country belongs to
sub_region	Sub-region the country belongs to
income_group	Income group as specified by the world bank in 2018
life_expectancy	The mean number of years a newborn would live if mortality patterns remained constant
income	GDP per capita (in USD) <i>adjusted for differences in purchasing power</i>
children_per_woman	Average number of children born per woman
child_mortality	Deaths of children under 5 years of age per 1000 live births

pop_density	Average number of people per km ²
co2_per_capita	CO2 emissions from fossil fuels (tonnes per capita)
years_in_school_men	Mean number of years in primary, secondary, and tertiary school for 25-36 years old men
years_in_school_women	Mean number of years in primary, secondary, and tertiary school for 25-36 years old women

Question 2

rubric={accuracy:1,quality:1,viz:2}

Python

1. I have included the data as a csv file in this directory so use `read_csv` from `pandas` to load the data directly from the URL and assign it a suitable variable name. Set the `parse_dates` parameter to `['year']` to ensure that Altair recognizes this columns as time data.
2. Now let's create a similar bubble chart to what you saw in the video:
 - Filter the dataframe to only keep observations from a single year, 1962. You can create a new data frame variable or perform the filtering directly as you pass the data to Altair. Dates can be matched as strings when filtering.
 - Use a circle mark to recreate the appearance of the plot in the video.
 - Encode the proper variables so that children per woman is on the x-axis, life expectancy on the y-axis, and so that the circles' color corresponds to their region, and the size reflects the population.

Don't worry about getting axis labels and sizes to be exactly like in the video, we will return to this code later in the lab to customize it.

```
In [2]: import pandas as pd

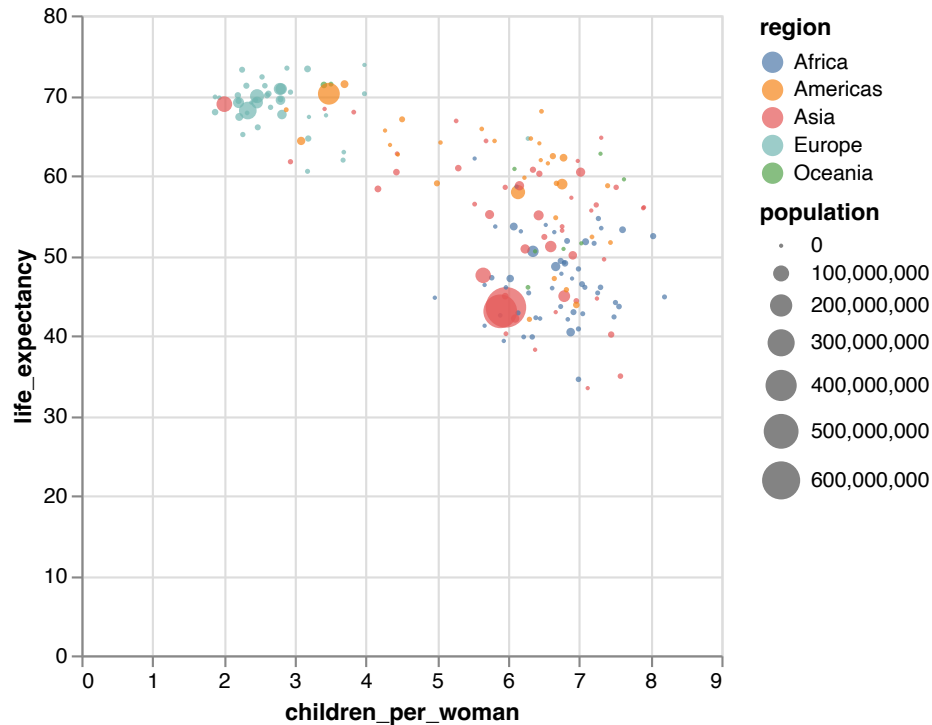
alt.renderers.enable('default')

url = "https://raw.githubusercontent.com/UofTCoders/workshops-dc-py/main/data/indicators.csv"
df = pd.read_csv(url, parse_dates=['year'])

chart_2 = alt.Chart(df[df['year'] == '1962']).mark_circle().encode(
    x='children_per_woman:Q',
    y='life_expectancy:Q',
    color='region:N',
    size='population:Q'
)
```

chart_2

Out[2]:



3. Education balance

A common misconception is that women around the world go to school many years less than men. Let's find out what the data actually says about this.

Question 3

```
rubric={accuracy:2,quality:1,viz:2}
```

Python

1. Compute a new column in your dataframe that represents the ratio between the number of years in school for women and men (calculate it so that the value 1 means as many years for both, and 0.5 means half as many for women compared to men).
2. Filter the dataframe to only contain value from 1970 - 2015, since those are the years where the education data has been recorded. Again you can either create a new variable or perform the filtering as you pass the data to the plotting function.
3. Create a line plot showing how the ratio of women's of men's years in school has changed over time. Group the data by income group and plot

the mean for each group.

4. Use layering to add a square mark for every data point in your line plot (so one per yearly mean in each group).

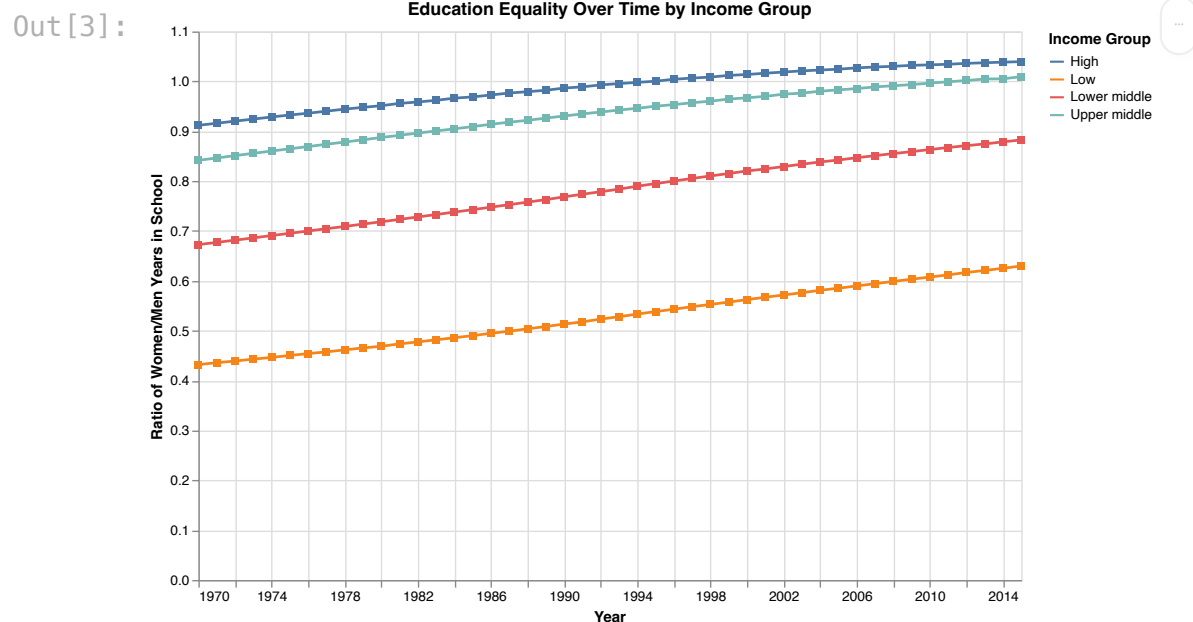
```
In [3]: df['edu_ratio'] = df['years_in_school_women'] / df['years_in_school_men']
edu_df = df[(df['year'] >= '1970-01-01') & (df['year'] <= '2015-01-01')]

line = alt.Chart(edu_df).mark_line().encode(
    x=alt.X('year:T', title='Year'),
    y=alt.Y('mean(edu_ratio):Q', title='Ratio of Women/Men Years in School'),
    color=alt.Color('income_group:N', title='Income Group')
)

points = line.mark_point(shape='square', filled=True)

chart_3 = (line + points).properties(
    width=600,
    height=400,
    title="Education Equality Over Time by Income Group"
)

chart_3
```



Question 3.1 (Optional)

rubric={accuracy:1}

Python

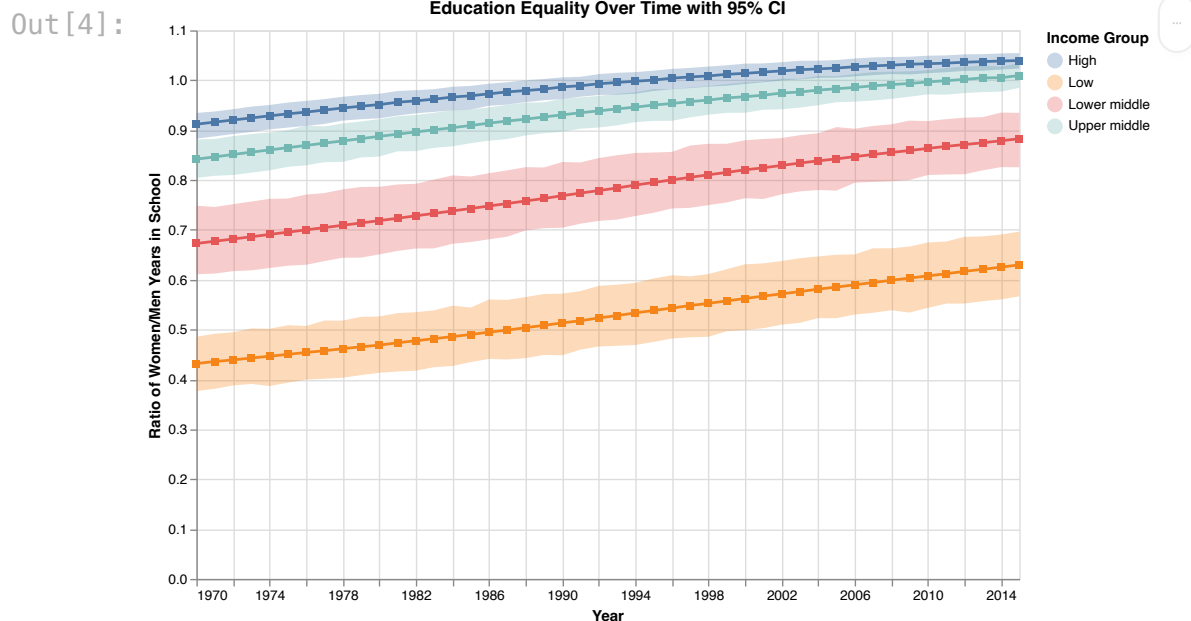
Add a [confidence interval band](#) to your line + square plot by assigning the plot

in the previous question to a variable name and then using layering to add the band. The default in the link above is a 95% bootstrapped confidence interval.

```
In [4]: band = alt.Chart(edu_df).mark_errorband(extent='ci').encode(
    x='year:T',
    y=alt.Y('edu_ratio:Q', title='Ratio of Women/Men Years in School'),
    color='income_group:N'
)

chart_3_1 = (band + line + points).properties(
    width=600,
    height=400,
    title="Education Equality Over Time with 95% CI"
)

chart_3_1
```



4. Family planning

Another common misconception is that saving the lives of children in low income countries will lead to overpopulation. Rather, lower child mortality is actually correlated with smaller family sizes. As more children survive, parents feel more secure with a smaller family size. Let's have a look in the data to see how this relationship has evolved over time.

In the plots we are going to make, it is important to note that it is not possible to tell causation, just correlation. However, in the [Gapminder](#) video library there are a few videos on this topic (including [this](#) and [this](#) one), discussing how reducing

poverty can help slow down population growth through decreased family sizes. Current estimates suggest that the world population will stabilize around 11 billion people and the average number of children per woman will be close to two worldwide in year 2100.

Question 4

rubric={accuracy:1,viz:2,reasoning:1}

Python

1. Filter the data to include only the years 1918, 1938, 1958, 1978, 1998, and 2018. To do this, you need to write out the full date strings, '1918-01-01' etc, or use `pd.to_datetime` with `format=%Y` on a list of the year integers only, up to you which one.
2. Use filled circles to make a scatter plot with children per women on the x-axis, child mortality on the y-axis, and the circles colored by the income group.
3. Facet your data into six subplots, one for each year laid out in 3 columns and 2 rows. To avoid taking too much space, set the width and height of the plots to suitable numbers.
4. Briefly describe your interpretation of the data. Does it support what was written in the introduction to this section of the lab? Why / why not?

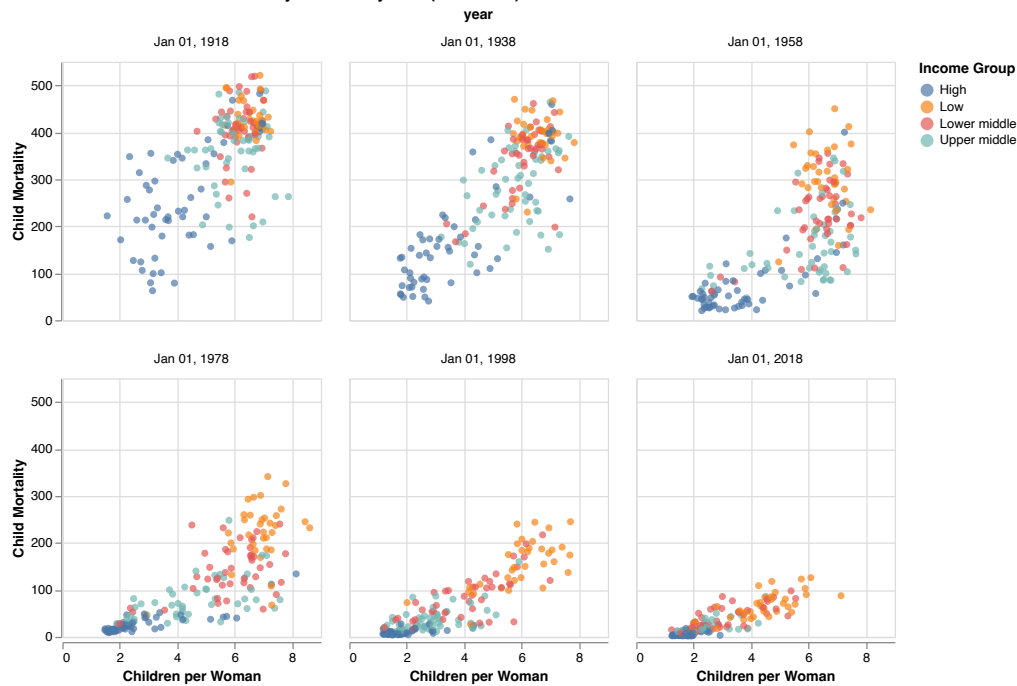
```
In [5]: years_to_keep = ['1918-01-01', '1938-01-01', '1958-01-01',
                        '1978-01-01', '1998-01-01', '2018-01-01']
df_q4 = df[df['year'].isin(years_to_keep)]

chart_4 = alt.Chart(df_q4).mark_point(filled=True).encode(
    x=alt.X('children_per_woman:Q', title='Children per Woman'),
    y=alt.Y('child_mortality:Q', title='Child Mortality'),
    color=alt.Color('income_group:N', title='Income Group'),
    tooltip=['country', 'year']
).properties(
    width=200,
    height=200
).facet(
    facet='year:T',
    columns=3
).properties(
    title='Correlation between Child Mortality and Family Size (1918-2
)

chart_4
```


Out [5]:

Correlation between Child Mortality and Family Size (1918-2018)



Yes, the visualization strongly supports the claim that lower child mortality is correlated with smaller family sizes. In the early facets (1918-1958), most countries have high child mortality and high fertility. As we move to 1978 and beyond, countries (especially higher-income ones first) shift toward the bottom-left of the graph. By 2018, nearly all countries have moved significantly toward lower child mortality and fewer children per woman, illustrating the global trend toward stabilizing populations as child survival rates improve.

5. Carbon dioxide emissions

CO2 emissions are often talked about in its relation to climate change. Let's explore the data to see which countries emit the most CO2 per capita and which regions have emitted the most in total over time.

Question 5

rubric={accuracy:1,quality:1,viz:2}

Python

1. Filter the data to include only the most recent year when `'co2_per_capita'` was measured (it is up to you how you find out

which year this is).

2. Use the data frame `nlargest` method to select the top 40 countries in CO2 production per capita for that year.
3. Since we have only one value per country per year, let's create a bar chart to visualize it. Encode the CO2 per capita as on the x-axis, the country on the y-axis, and the region as the color.
4. Sort your bar chart so that the highest CO2 per capita is the closest to the x-axis (the bottom of the chart). [Here is an example of how to sort in Altair.](#)

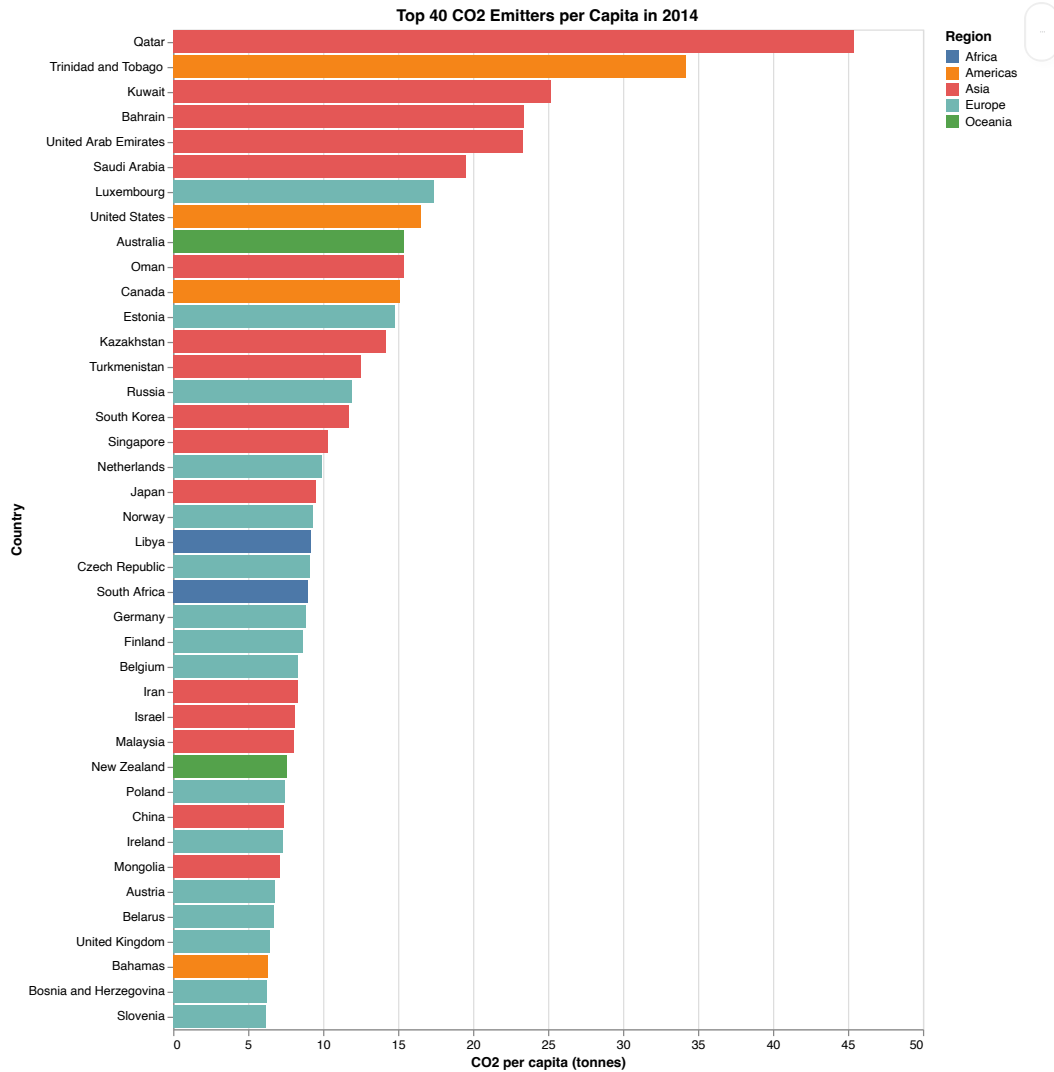
```
In [6]: recent_year = df[df['co2_per_capita'].notna()]['year'].max()

top_40_co2 = df[df['year'] == recent_year].nlargest(40, 'co2_per_capita')

chart_5 = alt.Chart(top_40_co2).mark_bar().encode(
    x=alt.X('co2_per_capita:Q', title='CO2 per capita (tonnes)'),
    y=alt.Y('country:N', sort='-x', title='Country'),
    color=alt.Color('region:N', title='Region')
).properties(
    title=f'Top 40 CO2 Emitters per Capita in {recent_year.year}',
    width=600
)

chart_5
```

Out[6]:



Question 5.1

rubric={accuracy:1,quality:1,viz:2}

Python

1. in addition to the co2 per capita, the total population also matter for a country's overall co2 emissions. compute a new column in your data set called `'co2_total'` which contains the total co2 emissions per observation.
2. plot this new column over time in an area chart, but instead of plotting one area for each country, plot one for each region which represents the sum of all countries co2 emissions in that region.

```
In [7]: df['co2_total'] = df['co2_per_capita'] * df['population']
```

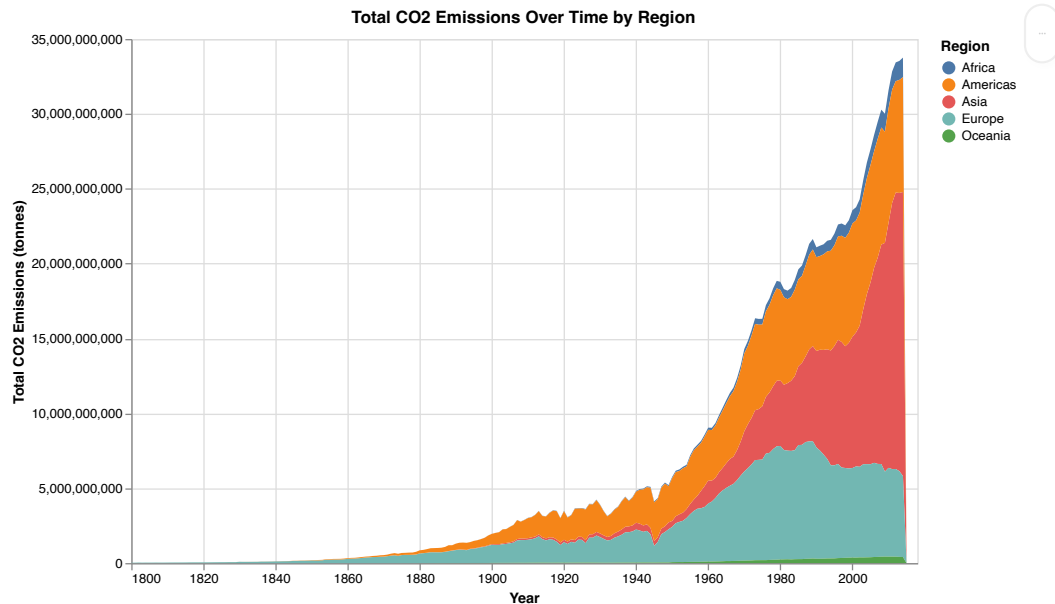
```

chart_5_1 = alt.Chart(df).mark_area().encode(
    x=alt.X('year:T', title='Year'),
    y=alt.Y('sum(co2_total):Q', title='Total CO2 Emissions (tonnes)'),
    color=alt.Color('region:N', title='Region'),
    tooltip=['region', 'year:T', 'sum(co2_total):Q']
).properties(
    width=600,
    height=400,
    title='Total CO2 Emissions Over Time by Region'
)

chart_5_1

```

Out[7]:



6. Income distribution

In his talk back in 2003, Rosling showed a projection of how the world income distribution would look like in 2015. Let's eyeball if the suggested trend was accurate.

Question 6 (Optional)

rubric={accuracy:1,viz:1}

Python

1. Wrangle your data to include the years 1979, 1991, 2003 and 2015.
2. Create a histogram (binned bar chart) of the income distribution with an appropriate number of bins.
3. Facet by year and make the plots smaller so that they fit in a single row.

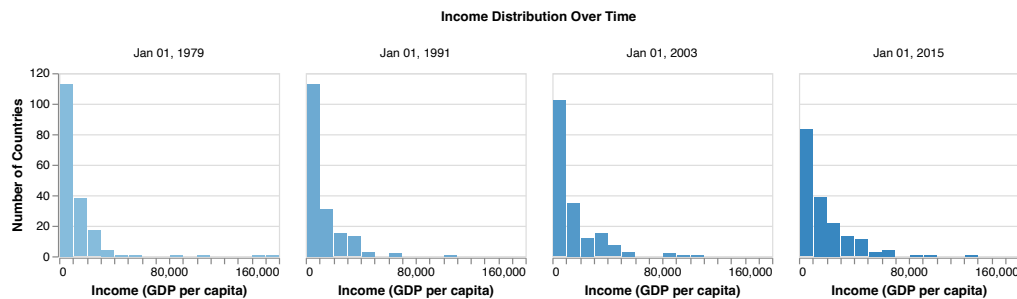
4. It is a little hard to tell if the data is exactly the same as the prediction since we are not using a log scale and a histogram instead of a density plot (we'll learn about these things later). But in general, briefly explain whether you think the trend is the same or not?

```
In [8]: target_years = pd.to_datetime(['1979-01-01', '1991-01-01', '2003-01-01', '2015-01-01'])
income_df = df[df['year'].isin(target_years)]

chart_6 = alt.Chart(income_df).mark_bar().encode(
    alt.X("income:Q", bin=alt.Bin(maxbins=30), title="Income (GDP per capita)",
    alt.Y("count()", title="Number of Countries"),
    color=alt.Color("year:T", legend=None)
).properties(
    width=180,
    height=150
).facet(
    column=alt.Column("year:T", title="Income Distribution Over Time")
)

chart_6
```

Out[8]:



7. Chart beautification

Let's make our charts from question 2 look more like the Gapminder bubble chart! Beautifying charts can take a long time, but it is also satisfying when you end up with a really nice looking chart in the end. We will learn more about how to create charts for communication later, but these parameters are usually enough to create basic communication charts and to help you in your data exploration.

Question 7

rubric={accuracy:2,quality:1,viz:1}

Python

1. Copy in your code from question 2.1 and confirm that your scatter plot is generated properly so that you didn't miss to copy anything.
2. Add a title of your choice to the chart.
3. Set the x-axis and y-axis scale so that they don't include zero and are zoomed in to the extent of the data instead.
4. Set proper titles for the axis and the legends, which include spaces instead of underscores and are capitalized.
5. Some of the dots are really hard to see because they are so small and it is a bit difficult to distinguish the changes in size as well. Let's make everything bigger and emphasize the size difference by using the `range` argument to `alt.Scale` (there is a lot of other things going on in this example, so just focus on how they specify `size`).
6. Enlarge the axis title font by finding and setting the `right` parameter of `.configure_axis`

```
In [9]: alt.renderers.enable('default')

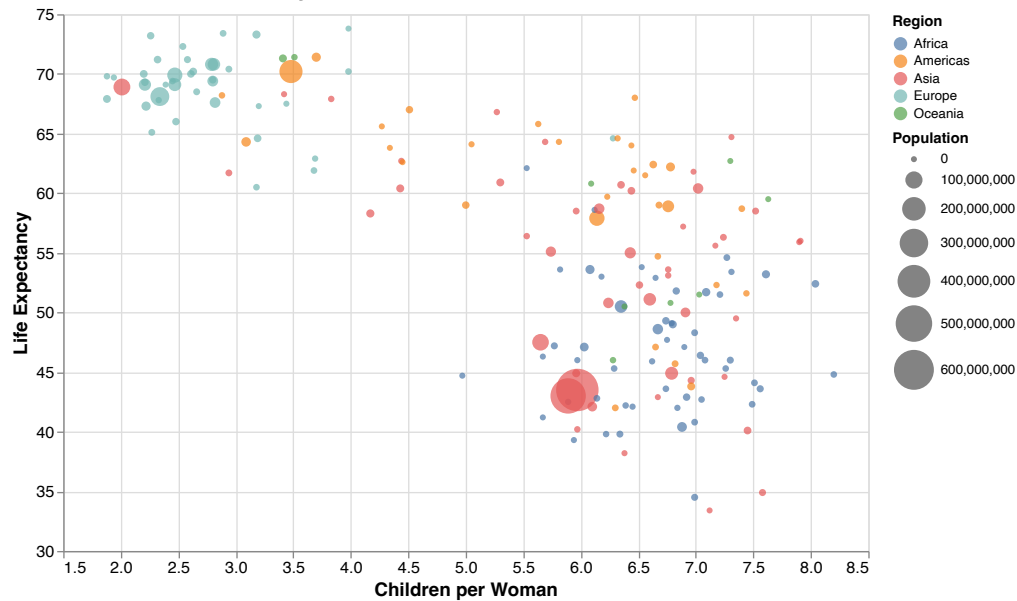
url = "https://raw.githubusercontent.com/UofTCoders/workshops-dc-py/main/data/ma
df = pd.read_csv(url, parse_dates=['year'])
plot_data = df[df['year'] == '1962-01-01']

chart_7 = alt.Chart(plot_data).mark_circle().encode(

    x=alt.X('children_per_woman:Q',
            title='Children per Woman',
            scale=alt.Scale(zero=False)),
    y=alt.Y('life_expectancy:Q',
            title='Life Expectancy',
            scale=alt.Scale(zero=False)),
    color=alt.Color('region:N', title='Region'),
    size=alt.Size('population:Q',
                  title='Population',
                  scale=alt.Scale(range=[20, 1000]))
).properties(
    title='Global Health and Fertility in 1962',
    width=600,
    height=400
).configure_axis(
    titleFontSize=14,
    labelFontSize=12
).configure_title(
    fontSize=18,
    anchor='start'
)

chart_7
```

Out [9]: **Global Health and Fertility in 1962**



Submission to Canvas

When you are ready to submit your assignment do the following:

1. Run all cells in your notebook to make sure there are no errors by doing
`Kernel -> Restart Kernel and Run All Cells...`
2. Commit and push all your work (including the .ipynb file) to GitHub.
3. Submit a link to your repo on Canvas (this has to be done only once per group).